Client Lifetime Value Prediction System

Technical Documentation

ML Pipeline for B2B Customer Revenue Prediction & CRM Insights

Random Forest • XGBoost • RFM Feature Engineering • sklearn Pipelines

Version	1.0
Date	May 14, 2026
Stack	Python 3.x, scikit-learn, XGBoost, pandas, numpy, joblib
Audience	ML Engineers, Data Scientists, Backend Developers

Table of Contents

1. Project Overview
2. Project Structure & Workflow
   - 2.1 File Overview
   - 2.2 End-to-End Pipeline
3. Dataset & Input Format
   - 3.1 Raw Input Schema
   - 3.2 Outlier Handling
4. Feature Engineering
   - 4.1 RFM Base Features
   - 4.2 Derived / Engineered Features
   - 4.3 Log-Transformed Features
5. Module Reference
   - 5.1 clean_dataset.py
   - 5.2 algo_choosing.py
   - 5.3 hypertuning.py
   - 5.4 test.py
6. Model Details
   - 6.1 Algorithm Comparison
   - 6.2 Hyperparameter Search
   - 6.3 Final Model Performance
7. CRM Inference & Business Outputs
8. Visualisations & Diagnostics
9. Setup & Running the Pipeline
10. Extending the System
11. Known Limitations & Future Work

1. Project Overview

Client Lifetime Value (CLV) Prediction is a machine learning pipeline that estimates the total future revenue a business can expect from each customer. Built as a service inside an AI-powered CRM platform, it processes transactional retail data, engineers rich behavioural features, trains and tunes a regression model, and then exposes actionable CRM insights — upsell opportunity tier, predicted CLV, and estimated cross-sell timing — for every customer.

Business Value

- Identify high-value customers for priority retention and upsell campaigns.
- Predict when and how aggressively to cross-sell based on expansion velocity.
- Rank customers objectively by predicted revenue for sales prioritisation.
- Provide explainable feature-importance insights to business stakeholders.
- Integrate as a microservice into a broader AI-powered CRM backend.

Final Model Performance (Random Forest, tuned)

0.9021	£477.74	£1,414	39.44%
R^2 Score	MAE	RMSE	MAPE

R^2 of 0.90 means the model explains 90% of variance in customer revenue — strong predictive power for a real-world retail dataset.

2. Project Structure & Workflow

2.1 File Overview

clean_dataset.py	Step 1. Loads raw Excel, cleans, engineers all features, clips outliers, saves cleaned_dataset.csv, and saves per-feature histogram plots + correlation heatmap.
algo_choosing.py	Step 2. Loads cleaned CSV, trains Random Forest and XGBoost inside sklearn Pipelines (with StandardScaler), evaluates both on real-scale and log-scale metrics, selects the best by R^2, saves the winner.
hypertuning.py	Step 3. Runs RandomizedSearchCV (20 iterations, 3-fold CV) over a Random Forest param grid on log-transformed target; saves the tuned model as clv_model_tuned.pkl; generates 4 diagnostic plots.
test.py	Step 4 / Inference. Loads the saved model, accepts a sample customer dict, predicts CLV, assigns upsell tier, and computes estimated cross-sell timing.
data/	Directory for all data artefacts: online_retail_dataset.xlsx (input), cleaned_dataset.csv (output of Step 1), saved_model/ (model pkl), plots/ (all diagnostic charts).
data/plots/	Auto-generated PNG charts: per-feature histograms, correlation heatmap, actual vs predicted, residual plot, residual distribution, feature importance.

2.2 End-to-End Pipeline

`clean_dataset.py`	**Load online_retail_dataset.xlsx**	Raw transactional Excel file (InvoiceDate, Quantity, Price, Customer ID, Invoice, StockCode)
`clean_dataset.py`	**Drop nulls, filter valid rows**	Remove rows with null Customer ID or non-positive Quantity/Price
`clean_dataset.py`	**Compute RFM + 7 engineered features**	Group by Customer ID → Recency, Frequency, Monetary, Unique_Products + 6 derived features
`clean_dataset.py`	**Clip outliers, add log features**	Hard caps on 4 columns; log1p transform on Frequency and Unique_Products
`clean_dataset.py`	**Save cleaned_dataset.csv + plots**	Customer-level feature matrix saved to data/; histograms and correlation heatmap saved to data/plots/

`algo_choosing.py`	**Train RF + XGBoost, compare metrics**	80/20 train-test split; log1p(Monetary) as target; both models evaluated on real + log scale
`algo_choosing.py`	**Select best model by R^2**	RandomForest wins (R^2=0.8999 vs 0.8761 for XGBoost)
`hypertuning.py`	**RandomizedSearchCV on RandomForest**	20 iterations × 3-fold CV over param_dist grid; scoring=r2
`hypertuning.py`	**Save tuned model + diagnostic plots**	clv_model_tuned.pkl via joblib; 4 diagnostic plots saved
`test.py`	**Load model, predict CLV, derive CRM signals**	Predicts log CLV → expm1 → Upsell tier + cross-sell timing

3. Dataset & Input Format

3.1 Raw Input Schema

The system expects an Excel file (online_retail_dataset.xlsx) containing transactional retail records. Each row represents one line item on one invoice.

Column	Type	Description
`Customer ID`	*Numeric / str*	Unique customer identifier. Rows with null Customer ID are dropped.
`InvoiceDate`	*datetime*	Date and time of the transaction. Parsed with pd.to_datetime().
`Invoice`	*str*	Invoice number. Used to count unique orders (Frequency).
`Quantity`	*int*	Number of units purchased. Rows with Quantity <= 0 are filtered out.
`Price`	*float*	Unit price. Rows with Price <= 0 are filtered out.
`StockCode`	*str*	Product code. Used to count unique products (Unique_Products).

*A derived column **TotalRevenue = Quantity × Price** is computed before aggregation. A **snapshot_date** = max(InvoiceDate) + 1 day is used as the reference point for Recency.*

3.2 Outlier Handling

After feature engineering, four columns are clipped at hard upper bounds to remove the influence of extreme outliers on model training:

Feature	Upper Clip	Rationale
`Monetary`	*50,000*	Caps extreme high-spend customers (bulk/wholesale anomalies).
`Items_Per_Order`	*1,000*	Caps unusually large single-order quantities.
`Expansion_Velocity`	*5*	Caps customers who purchased a very high diversity of products very quickly.
`Purchase_Consistency`	*60*	Caps inter-purchase standard deviation at 60 days.

4. Feature Engineering

4.1 RFM Base Features

The core features are derived from classical RFM (Recency-Frequency-Monetary) analysis, extended with product breadth:

Field / Parameter	Type / Value	Description
`Recency`	*int (days)*	Days since the customer's most recent purchase (lower = more recent).
`Frequency`	*int*	Number of unique invoices (orders) placed by the customer.
`Monetary`	*float*	Total revenue generated by the customer (sum of Quantity × Price). This is also the **prediction target** (log-transformed during training).
`Unique_Products`	*int*	Count of distinct StockCodes purchased — measures product breadth.

4.2 Derived / Engineered Features

Six additional features are engineered to capture customer behaviour beyond simple RFM:

Field / Parameter	Type / Value	Description
`Customer_Lifetime_Days`	*int (days)*	Days between first purchase and snapshot_date. Measures how long a customer has been active.
`Expansion_Velocity`	*float*	Unique_Products / (Customer_Lifetime_Days + 1). Rate at which a customer diversifies their product range. Proxy for engagement/growth potential.
`Purchase_Consistency`	*float (days)*	Standard deviation of inter-purchase intervals (days). Low value = regular buyer; high = erratic. Customers with only 1 purchase get 0.
`Items_Per_Order`	*float*	Total items purchased / Frequency. Average basket size per order.
`Purchase_Frequency_Rate`	*float*	Frequency / (Customer_Lifetime_Days + 1). Normalised purchase rate — accounts for how long the customer has been around.

4.3 Log-Transformed Features

Two features are log-transformed (numpy.log1p) to reduce right-skew before model training:

Feature	Formula	Purpose
Log_Frequency	*log1p(Frequency)*	Log of order count. Compresses the long tail of high-frequency customers.
Log_Unique_Products	*log1p(Unique_Products)*	Log of product count. Reduces skew from customers with very broad catalogues.

*The **target variable** (Monetary) is also log-transformed during training using np.log1p() and inverse-transformed with np.expm1() for evaluation and inference. This stabilises variance and improves model fit on skewed revenue distributions.*

Complete Feature Matrix (10 input features + 1 target):

Feature	Category	Notes
Recency	RFM base	Used directly
Frequency	RFM base	Used directly
Unique_Products	RFM base	Used directly
Customer_Lifetime_Days	Engineered	Used directly
Expansion_Velocity	Engineered	Used directly (clipped at 5)
Purchase_Consistency	Engineered	Used directly (clipped at 60)
Items_Per_Order	Engineered	Used directly (clipped at 1000)
Purchase_Frequency_Rate	Engineered	Used directly
Log_Frequency	Log transform	log1p(Frequency)
Log_Unique_Products	Log transform	log1p(Unique_Products)
Monetary	TARGET	log1p during training; expm1 for evaluation

5. Module Reference

5.1 clean_dataset.py

The data preparation module. Reads the raw Excel file, builds the customer-level feature matrix, and saves the cleaned CSV plus all exploratory plots.

Key function:

```
def get_final_b2b_matrix(df: pd.DataFrame) -> pd.DataFrame:
    """
    Input:  raw transactional DataFrame
    Output: customer-level feature matrix (one row per Customer ID)
    Steps:
      1. Drop null Customer IDs, filter Quantity>0, Price>0
      2. Compute TotalRevenue = Quantity * Price
      3. Set snapshot_date = max(InvoiceDate) + 1 day
      4. groupby('Customer ID').agg() → Recency, Frequency, Monetary, Unique_Products
      5. Engineer Customer_Lifetime_Days, Expansion_Velocity,
         Purchase_Consistency, Items_Per_Order, Purchase_Frequency_Rate
      6. Clip outliers on 4 columns
      7. Add Log_Frequency, Log_Unique_Products
      8. fillna(0) for any remaining NaN (e.g. single-purchase Purchase_Consistency)
    """
```

Execution block: *After calling the function, the script saves cleaned_dataset.csv, then loops over every column to save a histogram PNG, and finally saves a correlation heatmap.*

5.2 algo_choosing.py

Model selection module. Trains two candidate models inside sklearn Pipelines, evaluates both on six metrics, and saves the winner.

Models evaluated:

RandomForestRegressor	`n_estimators=400, max_depth=12, min_samples_split=4, min_samples_leaf=2, random_state=42`
XGBRegressor	`n_estimators=400, learning_rate=0.03, max_depth=5, subsample=0.8, colsample_bytree=0.8, random_state=42, verbosity=0`

Each model is wrapped in Pipeline([('scaler', StandardScaler()), ('model', model)]). Predictions are made in log-space, then inverse-transformed with np.expm1() before computing real-scale metrics (MAE, RMSE, R^2, MAPE).

Selection criterion:

```
if r2 > best_r2:
    best_r2 = r2
    best_model = pipeline   # RandomForest wins: R²=0.8999 vs 0.8761
```

5.3 hypertuning.py

Hyperparameter optimisation module for the winning RandomForest. Uses RandomizedSearchCV to efficiently explore a large parameter space.

Search configuration:

estimator	Pipeline(StandardScaler + RandomForestRegressor)
param_dist	n_estimators: [200,300,400,500] | max_depth: [8,10,12,15,None] | min_samples_split: [2,4,6,10] | min_samples_leaf: [1,2,4] | max_features: ['sqrt','log2',None]
n_iter	20 (random samples from param_dist)
scoring	'r2' (log-scale target)
cv	3-fold cross-validation
n_jobs	-1 (all available CPU cores)
random_state	42

*After fitting, the best estimator is evaluated on the held-out test set and saved as **clv_model_tuned.pkl** using joblib. Four diagnostic plots are generated.*

5.4 test.py

Inference script and CRM insight generator. Loads the saved model and converts a raw customer feature dict into business-readable outputs.

```
model = joblib.load('data/saved_model/clv_model_tuned.pkl')

# Build feature DataFrame matching training columns
df_sample = pd.DataFrame(sample_data)

# Predict
pred_log = model.predict(df_sample)
pred_clv = np.expm1(pred_log)[0]    # back to £ scale

# CRM signals
upsell = 'Low' if pred_clv < 500 else ('Medium' if pred_clv < 2000 else 'High')
cross_sell_days = 1 / (expansion_velocity + 1e-5)
```

6. Model Details

6.1 Algorithm Comparison

Metric	Random Forest	XGBoost	Winner
MAE (£)	481.39	498.00	**Random Forest**
RMSE (£)	1,430.79	1,591.16	**Random Forest**
R^2 Score	0.8999	0.8761	**Random Forest**
Log MAE	0.2992	0.2935	**XGBoost**
Log RMSE	0.4426	0.4166	**XGBoost**
MAPE (%)	39.70	34.12	**XGBoost**

*Random Forest is selected as the final model based on **real-scale R^2** (the primary business metric). XGBoost shows lower log-scale errors and MAPE, but R^2 on actual revenue is the most meaningful measure for CRM applications. The gap in MAPE is partially attributable to a few very low-revenue customers where percentage error inflates regardless of model.*

6.2 Hyperparameter Search

RandomizedSearchCV explores 20 random combinations from the parameter grid below. 3-fold CV on the training set is used to prevent data leakage from the test set.

Parameter	Search Values	Effect
`model__n_estimators`	`[200, 300, 400, 500]`	Number of trees in the forest.
`model__max_depth`	`[8, 10, 12, 15, None]`	Maximum depth per tree. None = unlimited.
`model__min_samples_split`	`[2, 4, 6, 10]`	Minimum samples required to split an internal node.
`model__min_samples_leaf`	`[1, 2, 4]`	Minimum samples in a leaf node. Larger = smoother model.
`model__max_features`	`['sqrt', 'log2', None]`	Number of features considered per split. None = all features.

6.3 Final Model Performance

After tuning, the best estimator achieves slightly improved metrics over the baseline Random Forest:

Metric	Value	Interpretation

MAE	*£477.74*	Average absolute error in predicted CLV (real currency scale).
RMSE	*£1,414.33*	Root mean squared error — more sensitive to large prediction errors.
R^2 Score	*0.9021*	Proportion of variance in customer revenue explained by the model.
MAPE	*39.44%*	Mean absolute percentage error. Elevated by very small-revenue customers.

7. CRM Inference & Business Outputs

test.py translates raw model predictions into three CRM-ready signals that can be consumed directly by a sales or marketing system:

Predicted CLV	**Formula:** np.expm1(model.predict(df_sample))[0] The predicted total lifetime revenue for the customer in the original currency scale (£).
Upsell Opportunity	**Formula:** CLV < 500 → 'Low' 500 ≤ CLV < 2000 → 'Medium' CLV ≥ 2000 → 'High' Categorical tier for CRM segmentation. High-tier customers should receive priority upsell and retention efforts.
Cross-sell Timing	**Formula:** 1 / (Expansion_Velocity + 1e-5) days Estimated days until the next optimal cross-sell opportunity. Customers who diversify products quickly (high velocity) should be targeted sooner.

Sample Inference (from test.py):

```
sample_data = {
    'Recency': [30],              # last purchased 30 days ago
    'Frequency': [5],             # 5 orders placed
    'Unique_Products': [40],      # bought 40 distinct products
    'Customer_Lifetime_Days': [200],
    'Expansion_Velocity': [0.2],  # diversifying product range
    'Purchase_Consistency': [10], # fairly regular buyer
    'Items_Per_Order': [150],
    'Purchase_Frequency_Rate': [0.025],
    'Log_Frequency': [np.log1p(5)],
    'Log_Unique_Products': [np.log1p(40)],
}

# --- Output ---
# Predicted CLV:              £2,350.00  (example)
# Upsell Opportunity:         High
# Estimated Cross-sell Timing: 5.0 days
```

Important: *All 10 features must be provided in the correct order and scale (Log_Frequency and Log_Unique_Products must be pre-computed before passing to the model). The StandardScaler inside the pipeline handles normalisation automatically.*

8. Visualisations & Diagnostics

All plots are auto-generated and saved to data/plots/ during pipeline execution.

Per-feature histograms (11 PNGs)	*Generated by: clean_dataset.py* One histogram per feature column. Reveals distribution shape, skew, and any remaining outlier presence after clipping. Saved as .png.
correlation.png	*Generated by: clean_dataset.py* Full correlation heatmap of the feature matrix (matplotlib imshow). Identifies highly correlated feature pairs that could indicate redundancy.
actual_vs_predicted.png	*Generated by: hypertuning.py* Scatter plot of actual vs. predicted CLV on the test set (real scale). A well-fitted model clusters points along the diagonal. Outliers reveal where the model struggles.
residual_plot.png	*Generated by: hypertuning.py* Scatter of predicted values (x-axis) vs. residuals (y-axis). Random horizontal scatter around zero indicates good model fit. Patterns suggest heteroscedasticity.
residual_distribution.png	*Generated by: hypertuning.py* Histogram of residuals. Should be approximately bell-shaped and centred at zero. Heavy tails indicate large errors on specific customer segments.
feature_importance.png	*Generated by: hypertuning.py* Horizontal bar chart of RandomForest feature importances sorted descending. Identifies which features most influence the CLV prediction.

*Feature importance analysis from the tuned model typically ranks **Monetary-related features (Frequency, Customer_Lifetime_Days, Log_Frequency)** as most predictive, as these directly correlate with historical spending behaviour.*

9. Setup & Running the Pipeline

Prerequisites

- Python 3.10 or higher
- online_retail_dataset.xlsx placed in data/
- No API keys required — fully local ML pipeline

Installation

```
cd 'client lifetime value prediction'

# Create virtual environment
python -m venv venv
source venv/bin/activate    # Windows: venv\Scripts\activate

# Install dependencies
pip install pandas numpy scikit-learn xgboost joblib matplotlib openpyxl
```

Running the Pipeline (in order)

Step 1: Data Cleaning

```
python clean_dataset.py
```

Reads online_retail_dataset.xlsx → saves data/cleaned_dataset.csv + data/plots/.png*

Step 2: Model Selection

```
python algo_choosing.py
```

Trains RF + XGBoost → prints comparison metrics → saves best model

Step 3: Hypertuning

```
python hypertuning.py
```

Runs RandomizedSearchCV → saves data/saved_model/clv_model_tuned.pkl + diagnostic plots

Step 4: Inference

```
python test.py
```

Loads tuned model → predicts CLV for sample customer → prints CRM insights

Updating Hardcoded Paths

Each script contains a hardcoded file_path / model_path pointing to the original Windows development machine. Update these before running on a new system:

```
# clean_dataset.py  — line 9
file_path = r"/your/path/to/data/online_retail_dataset.xlsx"
output_dir = r"/your/path/to/data"

# algo_choosing.py  — line 13
file_path = r"/your/path/to/data/cleaned_dataset.csv"

# hypertuning.py   — line 13
file_path = r"/your/path/to/data/cleaned_dataset.csv"
model_dir  = r"/your/path/to/data/saved_model"

# test.py           — line 5
model_path = r"/your/path/to/data/saved_model/clv_model_tuned.pkl"
```

10. Extending the System

Replace Hardcoded Paths with Config

Move all file paths to a config.py or .env file and load with python-dotenv or argparse. This makes the system portable without editing source files.

Add a REST API Layer

Wrap test.py in a FastAPI endpoint (POST /predict-clv). Accept a JSON body with the 10 feature values, return predicted CLV + upsell tier + cross-sell timing. Integrates seamlessly into the CRM backend.

Batch Inference

Instead of one sample_data dict, pass the entire cleaned_dataset.csv through the model to score all customers at once. Add the predictions back as columns and export as a ranked customer leaderboard.

Try Gradient Boosting Variants

Add LightGBM or CatBoost to algo_choosing.py for comparison. These often outperform XGBoost and Random Forest on tabular data with hypertuning.

Cross-Validation Instead of Single Split

Replace the single train_test_split with cross_val_score(cv=5) in algo_choosing.py for more robust model selection that is less sensitive to the random split.

Add Segmentation

After predicting CLV for all customers, apply K-Means or RFM quintile segmentation to group customers into tiers (Champions, Loyal, At-Risk, Lost) for targeted campaigns.

Persist Predictions to Database

In a production CRM, write the prediction results (customer_id, predicted_clv, upsell_tier, cross_sell_days) to a PostgreSQL or MongoDB collection after each run.

Schedule Periodic Retraining

Add a cron job or Celery task that re-runs the full pipeline (Steps 1–3) monthly as new transaction data arrives, keeping the model up-to-date.

11. Known Limitations & Future Work

Known Limitations

Hardcoded Windows Paths

All four scripts contain absolute Windows paths (C:\Users\...). These must be updated manually for any other environment. A config file or CLI args would solve this.

No Data Versioning

There is no mechanism to track which version of the dataset was used to train a given model .pkl file. Adding MLflow or DVC would solve this.

High MAPE (39%)

MAPE is inflated by customers with very low total spend (e.g. £5–£50), where even a small absolute error creates a large percentage error. Segment-specific models or a two-stage approach (classify low/high value first, then regress) could improve this.

No Temporal Validation

The train/test split is random, not time-based. In production, a time-based split (train on months 1–10, test on months 11–12) would better simulate real-world deployment.

Single Retail Dataset

The model is trained on one specific online retail dataset. Applying it to a different business context (B2B SaaS, subscription, etc.) would require retraining and likely different feature engineering.

No Missing Data Strategy

The pipeline uses a blanket fillna(0) for all missing values. Domain-specific imputation (e.g. median imputation for Purchase_Consistency) may improve accuracy for some customers.

Suggested Future Improvements

- Refactor all file paths into a central config.py or use argparse for CLI usage.
- Add time-based train/test split to simulate real-world deployment performance.
- Implement LightGBM / CatBoost in the model comparison step.
- Add K-Means customer segmentation on top of CLV predictions for CRM campaigns.
- Build a FastAPI microservice wrapping the inference pipeline for CRM integration.
- Add MLflow experiment tracking to log metrics, params, and artefacts per run.
- Add unit tests for get_final_b2b_matrix() and the upsell tier logic.
- Introduce a two-stage model: classify high/low value first, then regress CLV within each tier.

- Add SHAP values for per-customer explainability in the CRM dashboard.
- Implement periodic automated retraining as new transaction data arrives.

Documentation generated on May 14, 2026 • Client Lifetime Value Prediction v1.0